Part-2: Cross-Party Feature Entanglement Analysis

The Core Idea

Use the **Jacobian matrix** and **Singular Value Decomposition (SVD)** to measure how strongly one party's features influence another party's model in **Vertical Federated Learning (VFL)**.

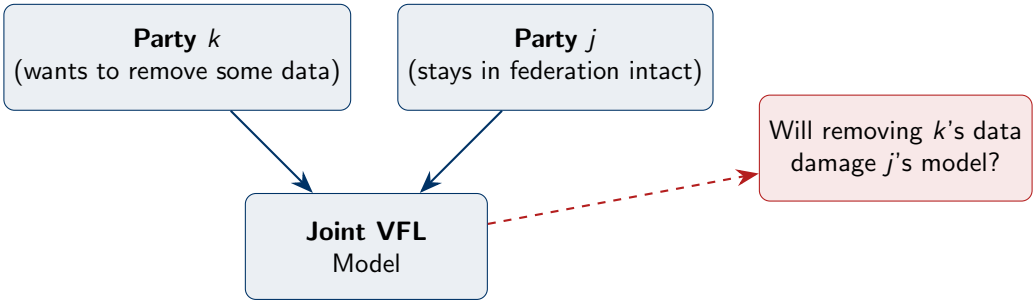
Novelty:

- No existing paper implements this *exact* method
- Uniquely combines:
  1. Cross-party Jacobian SVD
  2. VFL unlearning context
  3. Pre-unlearning diagnostic

Grounding:

- Every *individual* component has strong prior work:
  - Jacobian sensitivity analysis
  - SVD-based low-rank approx.
  - Feature entanglement in unlearning

The Problem

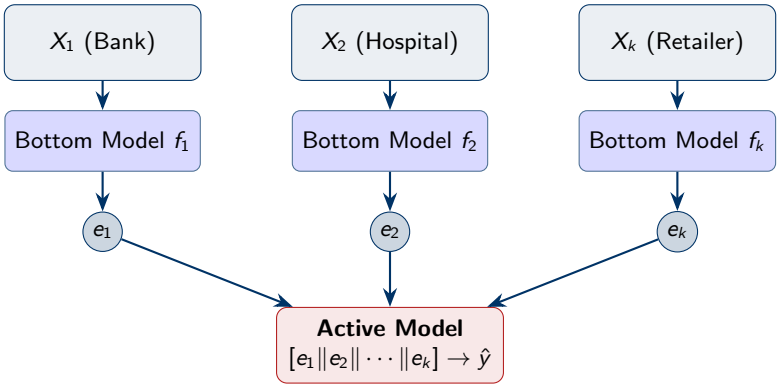


The Gap

Current VFL unlearning methods operate **blindly** — they remove data but give **no warning** about collateral damage to remaining parties.

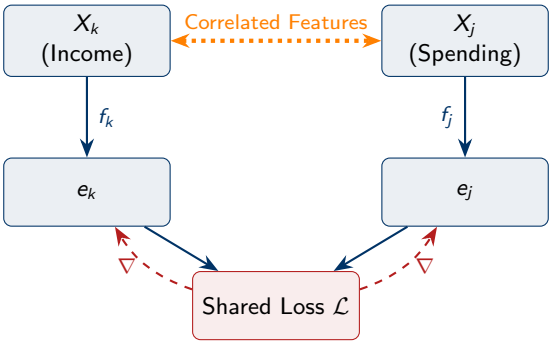
Background

VFL Architecture



Gradients flow back through all embeddings $\Rightarrow$ implicit coupling

Why Cross-Party Extension Is Needed



SensFL measures: $\partial e_k / \partial X_k$ (how my features affect *my* embedding)

We need: $\partial h_j^{(l)} / \partial X_k$ (how party k's features affect party j**'s** encoder)

Why Cross-Party Extension Is Needed

Three mechanisms of implicit coupling:

1. **Shared Loss:** All encoders optimize the same objective $\Rightarrow$ gradients couple representations
2. **Feature Correlation:** If X_k and X_j are statistically correlated (e.g., income & spending), encoder f_j exploits regularities involving X_k
3. **Active Model Adaptation:** The top model learns joint patterns from concatenated embeddings

Empirical Evidence (LASER-VFL)

Standard VFL suffers catastrophic degradation when features go missing *remain available*.

— *LASER-VFL, ICLR 2025*

Theoretical Foundation

Why the Jacobian? — Sensitivity as a Matrix

Definition

The Jacobian of function $f : \mathbb{R}^n \to \mathbb{R}^m$ is the $m \times n$ matrix of all first-order partial derivatives:

$$J = \frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Interpretation: Entry J_{ij} tells us how much output f_i changes when input x_j changes by a tiny amount.

In VFL: $J = \dfrac{\partial h_j^{(l)}}{\partial X_k}$ maps how party k's input features affect party j's hidden activations.

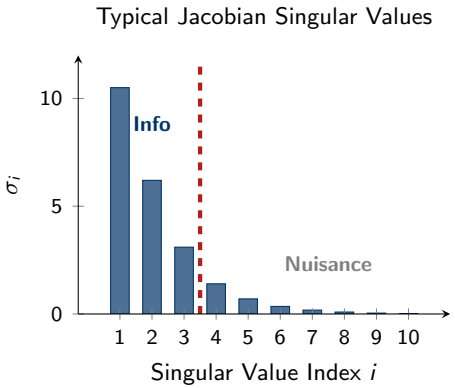
The Low-Rank Property of Neural Network Jacobians

Key Finding (Oymak et al., ICLR 2020)

Neural network Jacobians are **naturally low-rank**: a few large singular values dominate; most are near zero.

The spectrum decomposes into:

- **Information Space** — top singular values capturing meaningful data relationships
- **Nuisance Space** — small singular values capturing noise



SensFL: Jacobian Sensitivity in VFL

SensFL Formulation (CMES 2025)

For each client n, compute Jacobian norm:

$$J_n = \left\| \frac{\partial e_n}{\partial x_n} \right\|_2$$

Add as regularization $\Rightarrow$ reduces information content of embeddings $\Rightarrow$ defends against reconstruction attacks while maintaining accuracy ($< 5\%$ drop).

What SensFL validates:

- Jacobian captures meaningful feature-to-representation dependencies
- Practical in VFL settings

What SensFL misses:

- Only measures *same-party* sensitivity ($\partial e_n / \partial x_n$)
- Does not address *cross-party* influence ($\partial h_j / \partial X_k$)

Method Explained

Step 1: Compute the Cross-Party Jacobian

Formulation

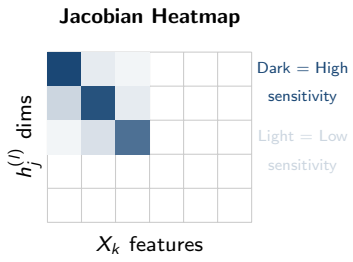
At each layer l of party j's model, compute:

$$J_{k \to j}^{(l)} = \frac{\partial h_j^{(l)}}{\partial X_k} \quad \in \mathbb{R}^{\dim(h_j^{(l)}) \times \dim(X_k)}$$

What each entry means:

$J_{ij}^{(l)}$ = how much a small change in the j-th feature of party k changes the i-th hidden activation of party j's layer l.

Analogy: A *sensitivity map* showing which of party k's features party j's encoder has learned to be sensitive to, even across organizational boundaries.



Step 2: SVD and the ρ Metric

Singular Value Decomposition

$$\frac{\partial h_j^{(l)}}{\partial X_k} = U\Sigma V^T \quad \text{where } \Sigma = \text{diag}(\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r)$$

The ρ Metric (Feature Influence Concentration)

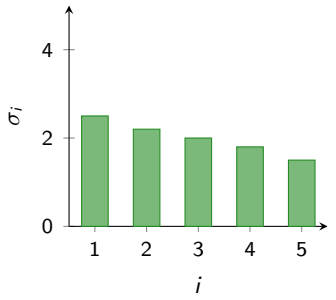
$$\boxed{\rho_{k,j}^{(l)} = \frac{\sigma_1}{\sqrt{\sum_i \sigma_i^2}} = \frac{\|\text{top singular value}\|_2}{\|\text{all singular values}\|_F}}$$

Range: $\rho \in [0, 1]$

- $\rho \to 0$: Influence diffusely spread (many small singular values) — low risk
- $\rho \to 1$: Influence concentrated in one direction — **high risk**

Interpreting ρ: Visual Intuition

Low $\rho \approx 0.35$

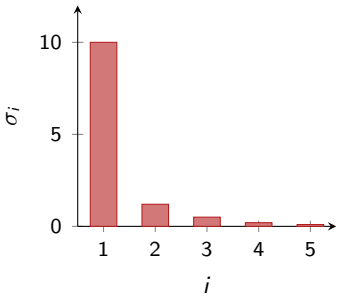


Broad, shallow coupling

Encoder can adapt to removal

$\Rightarrow$ Low unlearning risk

High $\rho \approx 0.92$



Concentrated, strong reliance

Encoder depends on one direction from X_k

$\Rightarrow$ High unlearning risk

Step 3: Low-Rank Approximation for Efficiency

Approximation

$$J \approx U_r \Sigma_r V_r^T \qquad \text{(keep only top-}r\text{ singular values/vectors)}$$

Memory Savings:

Representation	Memory
Full Jacobian	$O(d_1 \times d_2)$
Low-rank (r)	$O(r \times (d_1 + d_2))$

Example: $d_1 = 512$, $d_2 = 100$, $r = 10$

Full: 51,200 Low-rank: 6,120 (**12× savings**)

Why this works:

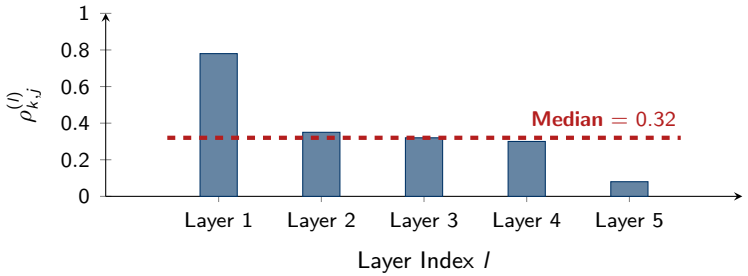
- Neural network Jacobians are *naturally* low-rank
- Small r (5–20) captures dominant dependencies
- The ρ metric only needs σ_1 and $\|\Sigma\|_F$ — perfectly captured by low-rank SVD

Step 4: Aggregate Across Layers

Layer Aggregation

For each (party k, party j) pair:

$$\text{Influence}_{k \to j} = \text{median}\left(\rho_{k,j}^{(l)}\right) \quad \text{over all layers } l$$



Layer 1 has anomalously high ρ (direct feature contact); Layer 5 has low ρ (vanishing gradients). Median captures the **typical** dependency.

Step 5: Classification Thresholds

Level	Threshold	Interpretation	Recommended Action
Low	$\rho < 0.15$	Minimal structured dependency on X_k	Unlearning proceeds safely
Medium	$0.15 \leq \rho < 0.40$	Moderate coupling; some degradation possible	Monitor party j's accuracy post-unlearning
High	$\rho \geq 0.40$	Strong, concentrated dependency	**Warn:** party j likely needs local retraining

Our Result

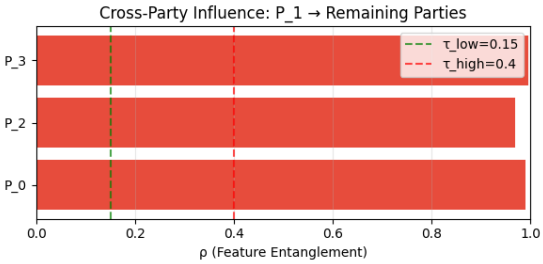


Figure 1: Cross party influence using Jacobian

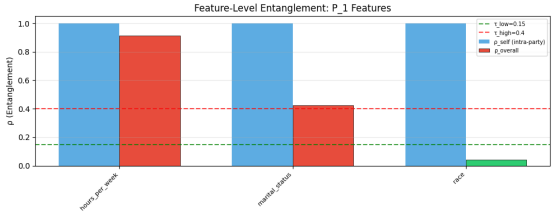


Figure 2: Feature entanglemnt between two parties using Jacobian

Challenges

Privacy Violation

Computing $\partial h_j^{(l)} / \partial X_k$ requires access to both party j's model internals **and** party k's input features — potentially violating VFL's privacy boundaries.

Computational Cost

Cross-party Jacobians for all (k, j) pairs across all layers:

$$\text{Cost} = O\big(K^2 \times L \times r \times n\big)$$

where K = parties, L = layers, r = approx. rank, n = samples.

Thank You